REFERENCE GUIDE

PowerGlove Vision Security

PowerGlove Vision documentation

2026 EDITION / IAIN BENNETT / MIT LICENSE

Use PowerGlove Vision on a trusted home or workshop network. This policy explains how to report a vulnerability and which protections the project expects pairing, networking, and shutdown code to maintain.

Supported code

Security fixes will be available on the current `main` branch and included in the next tagged release. Older snapshots may lack pairing, network, dependency, or shutdown protections and should be upgraded before troubleshooting them.

Reporting a vulnerability

Please do not publish credentials, pairing codes, tokens, private device configuration, or working exploit instructions in a public issue.

1. Open the repository's **Security** tab and look for private vulnerability reporting.
2. Submit a private report with the information below. Remove secrets from every attachment.
3. If private reporting is unavailable, open a minimal public issue asking for a private contact channel; include no exploit details or secrets.

Include these details in the private report:

- the affected commit or release;
- the UNO Q, RetroPie, browser, and network environment involved;
- concise reproduction steps and the observed result;
- the security boundary that was crossed;
- logs or screenshots after removing tokens, passwords, pairing codes, local addresses, and unrelated personal information.

This project does not currently offer a bug bounty or a guaranteed response time.

You can report controller-mapping, camera-compatibility, and game-profile problems in public issues after removing sensitive information from the logs.

Security model

PowerGlove Vision is designed for a trusted home or workshop network. The UNO Q performs hand tracking and sends virtual-controller state to RetroPie. RetroPie sends per-game profile changes back to the UNO Q. Neither device should be treated as an Internet-facing service.

The main protected assets are:

- the shared controller token;
- the UNO Q and RetroPie operating systems;
- the privileged `/dev/uinput` receiver;
- the physical pairing display and single-use PIN;
- the fixed-purpose UNO Q shutdown helper;

- the integrity of the App Lab installation ZIP, MediaPipe wheel, bundled or downloaded model, and Arduino dependencies.

The project does not attempt to protect a device after an attacker obtains root access, physical storage access, or control of the trusted local network and both paired hosts.

Pairing boundaries

The UNO Q and RetroPie share one random token of at least 16 characters. The active token belongs only in the UNO Q's private `data/device.json` and RetroPie's `/etc/powerglove/token`. It must not be committed, placed in a shell argument, stored in `launcher.json`, or included in a screenshot or log.

The recommended setup path uses a short-lived one-time code to authenticate the RetroPie pairing server over pinned TLS. Password pairing uses authenticated SSH. After the initial connection establishes trust, subsequent connections verify the saved remote host key. The password is not placed on the process command line.

Both browser pairing methods require you to open secure Setup, compare the browser certificate identity with the identifier on the UNO Q matrix, and enter the single-use PIN shown on the matrix before the token is released. This is a local certificate-pinning ceremony, not validation by a public certificate authority.

Pairing sessions limit how long a connection handshake can take and how long the pairing service remains available. Reusing a PIN, removing the physical display requirement, accepting pairing credentials over ordinary HTTP, or extending the listener indefinitely weakens the intended boundary and requires explicit security review.

Network exposure

Port	Protocol	Direction	Boundary
`55355`	UDP	UNO Q to RetroPie	Authenticated virtual-controller packets
`55356`	UDP	RetroPie to UNO Q	HMAC-authenticated profile commands and acknowledgements
`55357`	TCP/TLS	Pairing client to temporary server	Short-lived code-pairing exchange only
`8088`	HTTP	Browser to UNO Q	Local dashboard, public Help guides, diagnostics, and ordinary controls; no pairing credentials accepted
`8443`	HTTPS	Browser to UNO Q	Protected setup and pairing operations

Keep these ports on a trusted LAN. Do not configure router port forwarding, public reverse proxies, cloud tunnels, or Internet firewall exceptions for them. Guest Wi-Fi and untrusted shared networks are inappropriate unless the devices are isolated by firewall rules or a dedicated VLAN.

The profile-control brick publishes UDP `55356` and forwards packets to the worker on the private container network. It has no token or private data mount, runs without elevated privileges, and bounds packet sizes, pending exchanges, and reply lifetime. Authentication remains in the worker; the relay never creates an acknowledgement. A signed acknowledgement confirms queue admission, not completed camera startup or enabled gameplay delivery.

Controller and profile UDP traffic is authenticated but not encrypted. Anyone with access to the local network can observe packet timing and size even when they cannot create accepted input without the token. The dashboard can expose camera imagery and operational status to clients that can reach it, so network access to port `8088` is itself sensitive.

The Help library serves a fixed list of public Markdown guides and images from the installed application. It must not expose `data/`, the machine-specific cheat sheet, arbitrary filesystem paths, or pairing credentials. Markdown HTML is not executed, unsafe link schemes are rejected, and image requests are confined below `docs/images`.

The dynamic **This cabinet** page accepts only a validated hostname or IP from the browser `Host` header and combines it with `public_config()`. It may show local network addresses, ports, profile selection, camera selection, and whether pairing is configured, but it must never return the token value, passwords, private files, or arbitrary Host-header content.

Shutdown permissions

The web process does not receive general `sudo` permission. A root-owned systemd path unit watches one fixed file in the application data directory. A matching oneshot service deletes that file and requests a non-blocking Linux halt.

The dashboard route requires an explicit confirmation header and only creates the fixed request when the host installer has placed the private `.shutdown-enabled` marker. These checks reduce accidents and prevent command substitution; they do not make the dashboard safe for public network exposure. Anyone able to use the reachable dashboard may still cause a denial of service by shutting down the UNO Q.

A root-owned tmpfiles rule recreates only that fixed readiness marker during boot. It grants no command execution and does not change the container's privileges.

Keep the path unit, service unit, and tmpfiles rule owned by root, with file permissions set to `0644`. Do not replace the fixed `ExecStart` commands with user input, a shell string, or an arbitrary command runner. Remove or disable all three files if remote shutdown is not wanted.

Dependency and release integrity

- The App Lab installation ZIP is generated and verified; it is not maintained as a changing source-controlled binary.
- The custom MediaPipe wheel's provenance and checksum are recorded in `THIRD_PARTY_COMPONENTS.md`.
- Google's Hand Landmarker model is installed from the bundled copy, with its pinned download as a fallback only when that copy is absent. Both paths must match the expected SHA-256 digest before atomic installation; the package verifier also checks the bundled model and license text.
- Arduino library versions are pinned in `sketch/sketch.yaml`.
- GitHub Actions rebuilds and inspects documentation and the App Lab installation ZIP on every pull request and push to `main` or `dev`.

Changing a download URL, checksum, dependency source, pairing primitive, network binding, file permission, or privileged service requires focused review and corresponding tests and documentation.

Paired game editing and gesture tuning

The separate RetroPie Games service listens on TCP `55358`. Only the paired UNO proxy uses it; browsers call the UNO website. Requests and replies use a distinct HMAC-authenticated protocol. Server challenges expire after fifteen seconds and are consumed once. The shared token never goes to the browser. This protects message integrity; the LAN transport does not encrypt ROM filenames.

The service accepts only registry reads, validated writes, and restoration. File locations come from the administrator's launcher configuration, never from browser input. Writes use revision checks, atomic replacement, and a previous valid backup. The service has bounded document sizes, pending challenges, and socket timeouts; it runs separately from

controller input delivery. Its systemd unit confines writes to the configured registry directory and removes device access and capabilities.

The new Games and Tune browser actions require JSON, an explicit action header, and matching Origin when supplied; cross-site browser requests are rejected. They retain the existing trusted-LAN administration model, not per-user accounts. Personal tuning contains numerical thresholds only. Measurements are held briefly in memory, previews expire with the owning session, and camera images are not saved. Tuning suppresses controller delivery even if a game launches or another Dashboard requests input. Saved settings are validated and atomically replaced.

Documentation screenshots

Documentation screenshots blur the complete camera image before capture. Keep controls legible, but never publish unblurred camera frames or screenshots that contain passwords, private tokens, or pairing codes. The reference images show the interface; they are not saved gesture recordings.